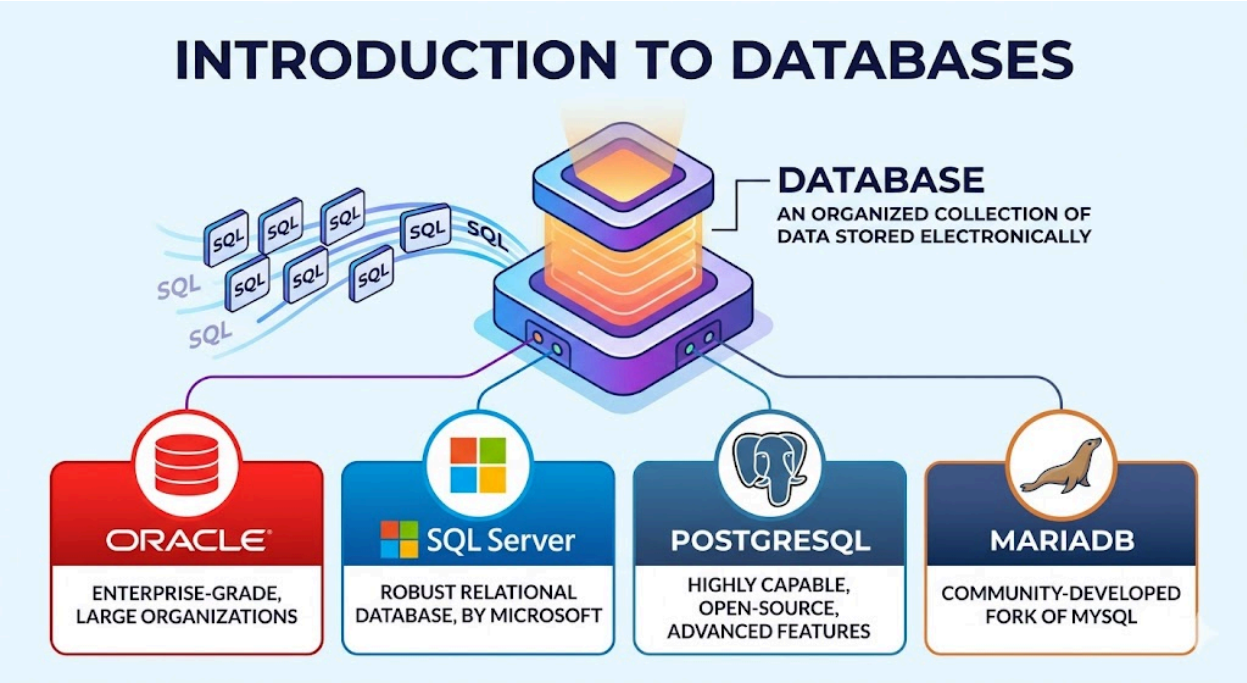
Day 1: Introduction to Databases & CRUD in SQL



1. Introduction to Databases

Before diving into the code, it is essential to understand where our data lives.

What is a Database?

A database is an organized collection of data that is stored and accessed electronically. It allows for the efficient retrieval, insertion, and management of information through structured queries using **SQL** (Structured Query Language).

Popular SQL Database Systems

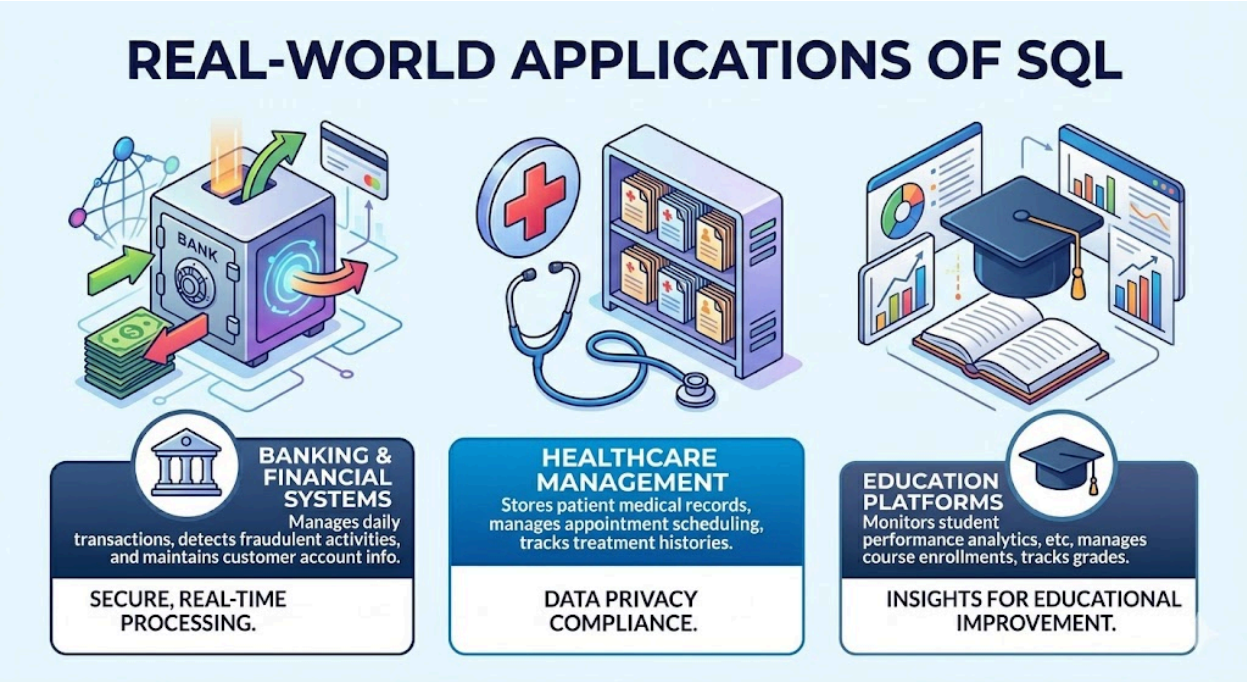
Depending on the company or project, you might encounter different database systems. Some of the most popular include:

Oracle: An enterprise-grade database designed for large organizations.

SQL Server: Microsoft's robust relational database platform.

PostgreSQL: A highly capable open-source database known for its advanced features.

MariaDB: A community-developed fork of MySQL.



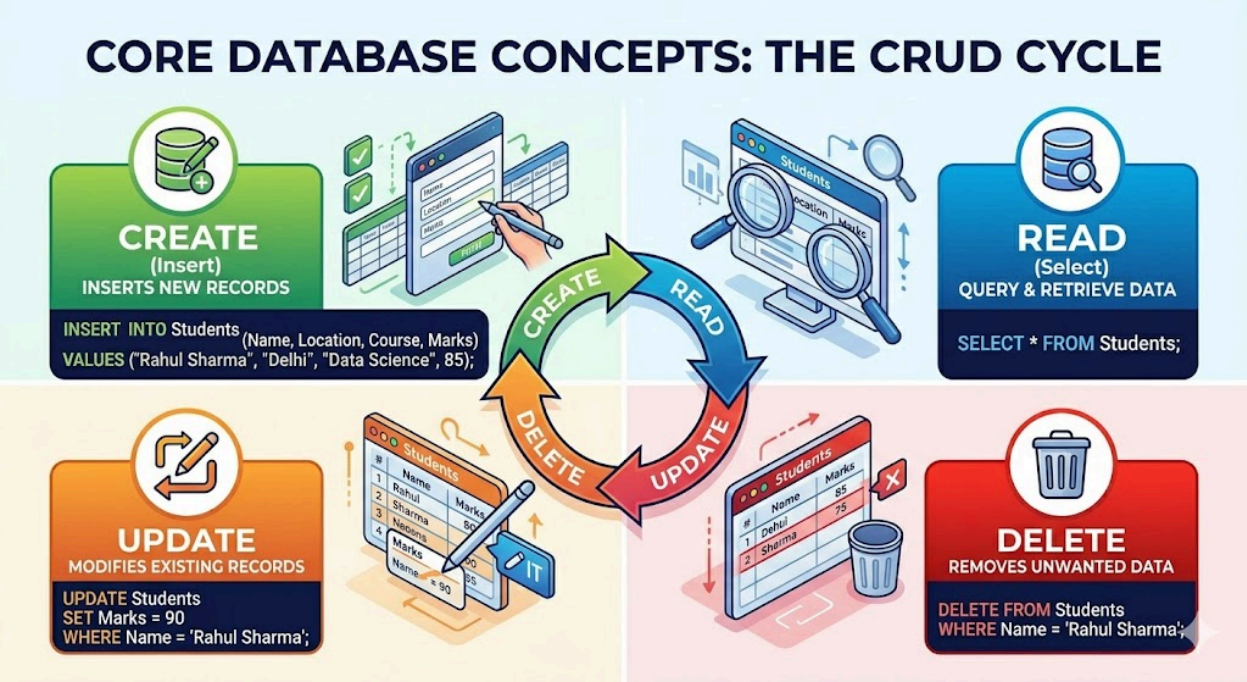
2. Real-World Applications of SQL

SQL is the backbone of modern data management across almost every industry. Here is how it is used in the real world:

Banking and Financial Systems: Manages millions of daily transactions, detects fraudulent activities, and maintains customer account information securely with real-time processing.

Healthcare Management: Stores patient medical records, manages appointment scheduling, tracks treatment histories, and ensures data privacy compliance.

Education Platforms: Monitors student performance analytics, manages course enrollments, tracks grades, and provides insights for educational improvement.



3. Core Database Concepts: CRUD Operations

CRUD represents the four basic operations that define data manipulation in databases. These fundamental operations form the very foundation of all database interactions.

Below are the concepts paired with standard SQL code examples so you can see how they look in practice.

1. Create (Insert) This operation inserts new records into your tables.

SQL

- `- Creating a new student record in the databaseINSERT INTO Students (Name, Location, Course, Marks)`
  `VALUES ('Rahul Sharma', 'Delhi', 'Data Science', 85);`

2. Read (Select) This operation is used to query and retrieve data.

SQL

- `- Retrieving all columns for every student in the tableSELECT  FROM Students;`

3. Update This operation modifies existing records.

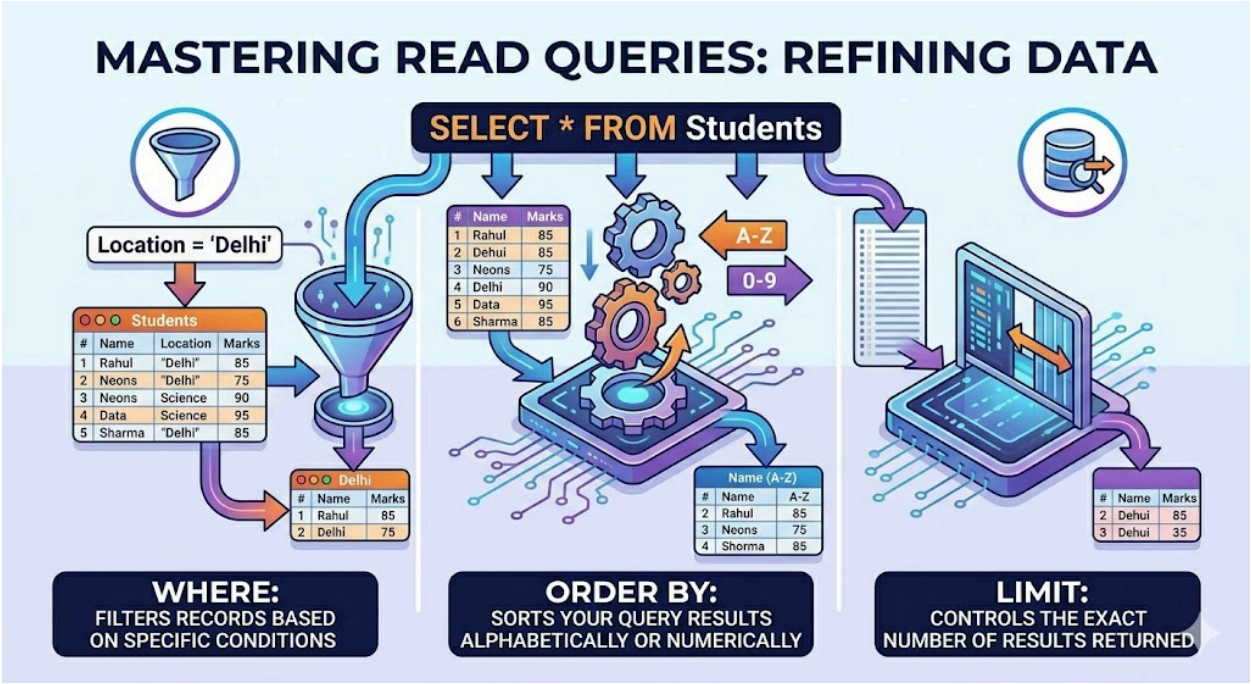
SQL

- `- Updating a student's marks`
  `UPDATE Students`
  `SET Marks = 90WHERE Name = 'Rahul Sharma';`

4. Delete This operation removes unwanted data.

SQL

- `- Deleting a student recordDELETE FROM Students`
  `WHERE Name = 'Rahul Sharma';`



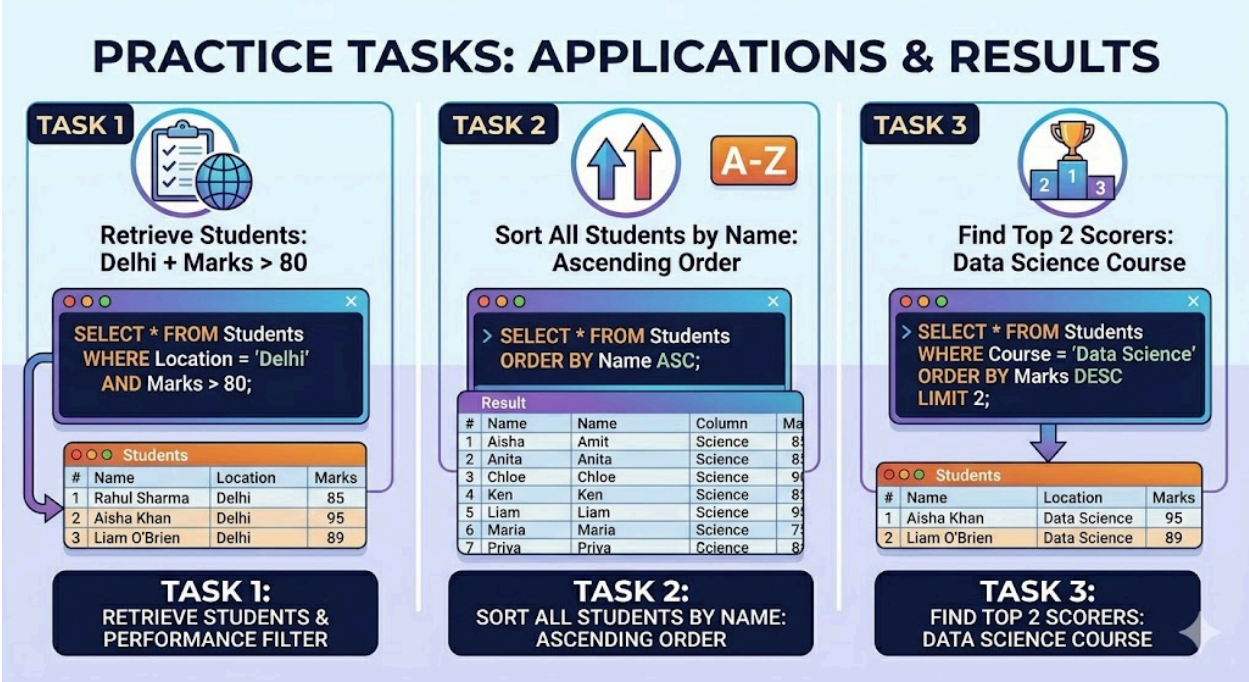
4. Additional Query Operations

To make your "Read" operations more powerful and precise, SQL uses specific keywords to organize and filter your data.

ORDER BY: Sorts your query results alphabetically or numerically.

WHERE: Filters records based on specific conditions you set.

LIMIT: Controls the exact number of results returned by the database.



5. Practice Tasks (Hands-On Application)

To truly learn SQL, you need to write it. We recommend using an online SQL editor like **DB Fiddle** or **SQLite Studio** to execute your queries and experiment with different commands.

Apply your SQL knowledge to solve these practical scenarios using the `Students` table.

Task 1: Location & Performance Filter

Write a query to retrieve all students from Delhi who have scored marks above 80. Combine location and performance criteria using appropriate `WHERE` conditions.

Solution:

SQL

```
SELECT * FROM Students
WHERE Location = 'Delhi' AND Marks > 80;
```

Task 2: Alphabetical Sorting

Create a query that sorts all students by name in ascending order from A to Z. Use `ORDER BY` to arrange the results alphabetically.

Solution:

SQL

```
SELECT * FROM Students
ORDER BY Name ASC;
```

Task 3: Top Performers

Find the top 2 scorers specifically in the 'Data Science' course. Combine `ORDER BY` with `LIMIT` to get the highest performers efficiently.

Solution:

SQL

```
SELECT * FROM Students
WHERE Course = 'Data Science'
ORDER BY Marks DESC
LIMIT 2;
```